# Anomaly Detection in Operating System Logs with Deep Learning-Based Sentiment Analysis

Hudan Studiawan, Ferdous Sohel, *Senior Member, IEEE*, and Christian Payne

**Abstract**—The purpose of sentiment analysis is to detect an opinion or polarity in text data. We can apply such an analysis to detect negative sentiment, which represents the anomalous activities in operating system (OS) logs. Existing methods involve manual searching, predefined rules, or traditional machine learning techniques to detect such suspicious events. In this article, we propose a novel deep learning-based sentiment analysis technique to check whether there are anomalous activities in OS logs. Log messages are modeled as sentences and we identify the sentiments using the gated recurrent unit (GRU) networks. OS log datasets inherently have a class imbalance in the sense that the number of negative sentiment is much lower than that of the number of positive ones. In order to address the class imbalance, we build a GRU layer on top of a class imbalance solver using the Tomek link method. Experimental results demonstrate that the proposed method can detect anomalous events in OS logs with an overall F1 and accuracy of 99.84 and 99.93 percent, respectively.

**Index Terms**—Anomaly detection, sentiment analysis, deep learning, operating system logs, class imbalance

## 1 Introduction

Sentiment analysis is commonly applied to social media or product review data. It enables us to analyze user preferences in various contexts. The purpose of sentiment analysis is to detect the opinion or polarity in text data [1]. In this case, the sentiment can be positive, neutral, or negative. The most popular data to be analyzed in terms of sentiment are social media data [2], [3] and customers' product reviews [4]. We are able to identify what people like or dislike by analyzing the sentiments. The results can be used for evaluating the products.

Because sentiment analysis can be applied to text data, we can use it to analyze operating system (OS) logs. An OS records its activities in log files. The recorded messages are not only activities conducted by the system itself but also show interactions with users. The OS also logs various activities including attempts to obtain unauthorized access to a server. The messages in OS logs have rich content and contain both positive and negative sentiments. Therefore, it is important to examine the OS logs using sentiment analysis. In addition, sentiment-based detection can support the security analysis or forensic investigation of OS logs, especially using deep learning as suggested in [5].

The system administrator who has responsibility for the computer security needs to inspect the log files to check for anomalous messages or irregularities. The checking process can take a significant amount of time if launched by a manual command in the terminal [6]. The use of predefined rules for anomalous activities does not provide flexibility as the various messages continue to be saved in log files. The existing log management tool, such as Splunk [7], requires input threshold from the users to detect anomalies with common statistics-based methods, namely the Z-score and the interquartile range (IQR). The static threshold cannot adapt with dynamically changing log entries in various cases [8]. Another technique is to use traditional machine learning methods [9]. However, the accuracy of this method is not very high.

To detect the log anomalies in a production OS, we propose to use sentiment analysis in the OS log files. We consider this problem as two classes sentiment analysis, specifically positive and negative sentiments. We argue that the detection of negative sentiments is equivalent to identifying anomalous activities from OS log messages. On the other hand, the positive sentiment is the normal or other regular messages recorded in an OS log file.

An illustration to support this argument is provided in Fig. 1. We highlight the positive sentiment of log messages in green and the negative in red. The negative sentiments that emerged in this example are "invalid user" and "user unknown". These types of entries need to be investigated further. Furthermore, sentiment analysis mainly uses machine learning to detect positive or negative content. On the other hand, deep learning has been applied and has gained popularity as a means of analyzing sentiment and it has been shown to produce better accuracy than traditional machine learning methods [10].

We use a deep learning technique that provides both high accuracy and flexibility in regard to previously unseen data. Specifically, we use a gated recurrent unit (GRU) networks [11] to detect the sentiment in OS log messages. In real-life OS logs, the number of negative messages is much smaller than the positive ones, which leads to class imbalance. We

- *Hudan Studiawan is with the Discipline of Information Technology, Media & Communications, Murdoch University, Perth, WA 6150, Australia, and also with the Department of Informatics, Institut Teknologi Sepuluh Nopember, Surabaya 60111, Indonesia. E-mail: hudan.studiawan@murdoch.edu.au.*
- *Ferdous Sohel and Christian Payne are with the Discipline of Information Technology, Media & Communications, Murdoch University, Perth, WA 6150, Australia. E-mail: {f.sohel, c.payne}@murdoch.edu.au.*

*Manuscript received 29 Sept. 2019; revised 14 Oct. 2020; accepted 10 Nov. 2020. Date of publication 13 Nov. 2020; date of current version 1 Sept. 2021. (Corresponding author: Hudan Studiawan.)*
*Digital Object Identifier no. 10.1109/TDSC.2020.3037903*

...
Feb 6 15:04:39 victoria login[1990]: pam_unix(login:session): session opened for user root by LOGIN(uid=0)
Feb 6 15:04:39 victoria login[2042]: ROOT LOGIN on 'tty1'
Feb 6 15:16:20 victoria sshd[2085]: Invalid user ulysses from 192.168.56.1
Feb 6 15:16:20 victoria sshd[2085]: Failed none for invalid user ulysses from 192.168.56.1 port 34431 ssh2
Feb 6 15:16:24 victoria sshd[2085]: pam_unix(sshd:auth): check pass; user unknown
…

Green: positive sentiment
Red: negative sentiment = anomalous activites in log messages

Fig. 1. An illustration of a section of an OS authentication logs with negative and positive sentiments.

consider this issue to achieve a balance between the two sentiment classes by using the Tomek link method [12]. The balancing will produce a better deep learning model; therefore, more accurately detect anomalous activities as the minority class.

*Contributions*. In general, the contributions of this work are as follows:

1) To the best of our knowledge, this paper is the first work to use sentiment analysis for identifying anomalous activities in OS logs. Detecting negative sentiments can be considered as a new way of detecting irregularities in OS logs.
2) We consider class imbalance in OS log data by applying the Tomek link method.
3) We build a GRU layer to detect sentiment analysis of log messages. This layer is built on top of the Tomek link method and a word embedding layer.
4) To enable reproducibility of this research, we provide the source code implementation of the proposed method, the preprocessed and labeled system log datasets, and the trained model on a GitHub repository.[1] The model can be readily used to detect anomalous activities in an OS log file. Finally, we name the proposed method *pylogsentiment*.

The paper is organized as follows. Section 2 describes the related work on log anomaly detection, broad sentiment analysis using deep learning, the use of sentiment analysis in event logs, and the class imbalance problem in sentiment analysis. Threat model and assumptions used in this paper are provided in Section 3. Section 4 explains the proposed method named *pylogsentiment*. Experiment results and analysis are reported in Section 5. In Section 6, we give the conclusion of this work.

## 2 Related Work

This section reviews the related work on anomaly detection in log files. Moreover, we briefly describe the deep learning for sentiment analysis and its application in event log data. Subsequently, we discuss related work on class imbalance in the context of sentiment analysis.

### 2.1 Anomaly Detection in Event Log Data

An anomaly is the data pattern that are not similar to the normal data behavior [13]. In the case of log data, anomalies are entries that repeatedly appear (e.g., brute force attacks) or rarely occur (e.g., a particular service stopped unexpectedly). The system administrator has to be aware of these types of suspicious events.

Broadly there are three types of anomalies, namely point, collective, and contextual anomalies [13]. Point anomaly is an individual instance that is anomalous to the rest of the data. Collective anomaly is when several data occur together as a collection and appear to be anomalous, while its individual instance may not be an anomaly. The last type is the contextual anomaly when the data appear in a specific context, which has been defined regarding each particular problem.

He *et al*. [14] reviewed several supervised and unsupervised methods for anomaly detection in publicly available logs. The unsupervised approaches discussed are isolation forest [15] and principal component analysis (PCA) [16]. The supervised methods include the decision tree [17], logistic regression [18], and support vector machines (SVM) [19]. The paper [14] reported that the supervised methods provide higher recall and precision values than the unsupervised ones because the former category has been trained on the provided datasets.

At an OS level, one can detect anomalies using statistical predictors and a safety margin [8]. They [8] analyze various features, such as the system call errors, OS signals, and various device timeouts. A lower and an upper anomaly thresholds have to be calculated and trained based on the mean and standard deviation of the log data.

Bitton and Shabtai [20] provided another work for OS level anomaly detection. They focused on a remote desktop protocol installed in electronic flight bag servers. Anomaly detection acts as a network-based intrusion detection system. This approach is considered as a fine-grained model because it comprises multiple anomaly detection models such as k-means clustering and the cluster-based local outliers factor (CBLOF).

Recent research in log anomaly detection has started to adopt deep machine learning techniques because they show a significant improvement in the performance. For instance, DeepLog [21] models log entries as a sequence and then trains the normal sequence with the long short-term memory (LSTM) networks. When there are data that do not conform with the normal model, they are defined as anomalies. Note that DeepLog is mainly designed for collective anomaly detection, which is another type of anomaly detection. It requires a sequence of entries as the input. However, DeepLog can also detect point anomaly. It accepts a history log sequence as input and checks if the next single log entry is normal or not by comparing the actual log that appears in its prediction.

Another LSTM-based deep learning model is used to detect anomalies in cloud operation log files [22]. Recurrent

1. https://github.com/studiawan/pylogsentiment

neural networks (RNN) have been applied along with attention-based deep learning to capture the context between words in log entries [23]. Unlike DeepLog [21], the latter method was trained using both normal and anomalous datasets.

The aforementioned works look into the statistical properties of log data or modeling log entries as sequences. Unlike the existing approaches, we propose to detect anomalies based on the sentiment of the log messages. We detect a log entry as an anomaly when the message in the entry shows a negative sentiment.

### 2.2 Deep Learning for Sentiment Analysis

Many popular deep learning methods have been used to address sentiment analysis, such as convolutional neural networks (CNN) [24], long short-term memory (LSTM) [25], and gated recurrent unit (GRU) [26]. Note that the last two architectures are extensions of recurrent neural networks (RNN) [27]. The advantage of deep learning compared to traditional machine learning is that deep learning does not need feature definition of the text data. Another benefit of deep learning is that it provides an easier model development as the developer needs to give only the input data and basic initial parameters for training. In the testing phase, the users only input the data and they will be processed based on the models generated in the learning phase.

dos Santos and Gatti [28] examined short text, especially Tweet data, at both character-level and sentence-level and then process them with CNN. Severyn and Moschitti [29] used CNN to train the sentiment model. The difference is that they first initialized the parameter weights of the convolutional neural network to increase the accuracy. Socher *et al.* [30] introduced a sentiment treebank and trained them on the recursive neural tensor network. Moreover, Araque *et al.* [3] used ensemble learning techniques for sentiment analysis with deep learning.

Kim [24] proposed CNN along with pre-trained word embedding to detect sentiment polarity. It shows good accuracy against various benchmarks. Furthermore, Radford et al. [31] used LSTM in the byte-level representation of text. Another work employed hierarchical LSTM to detect sentiment and then considered user and product attention [4]. For a comprehensive review of sentiment analysis by means of deep learning, the reader is referred to [10].

### 2.3 Sentiment Analysis in Event Log Data

Sentiment analysis in log data is not so popular if we compare it to social media or product reviews. In [32], sentiment analysis was applied to web query logs to detect political sentiment, specifically right or left wing. A sentiment polarity can be employed to identify developer emotion in commit logs on public GitHub software repositories [33]. The method used in that research is the SentiStrength, which is based on a dictionary of sentiment terms and machine learning [34]. Using the SentiStrength, the authors concluded that a project with a greater number of team members tends to have more positive sentiment.

Similar research [35] showed that the large number of files changed in a software project tends to produce negative sentiment in commit logs. The method used was also SentiStrength. These researches [33], [35] reveal that the commit at the beginning of the week, specifically Monday and Tuesday, tend to have negative polarity.

However, none of the aforementioned works in event log data applied deep learning to sentiment analysis. In addition, none of these research addresses the problem in event logs for computer security purposes. We conduct research on the impact of sentiment analysis on the system logs in order to detect anomalous activities in them.

### 2.4 Class Imbalance in Sentiment Analysis

The imbalance between the positive and negative classes in sentiment analysis has been considered in several works. Three main techniques are used to address the class imbalance, namely under-sampling, over-sampling, and a combination of these two. Mountassir *et al.* [36], [37] compared under-sampling methods, specifically random under-sampling, remove similar, remove furthest, and remove by clustering. Subsequently, the authors used traditional machine learning such as naïve Bayes, support vector machines, and k-nearest neighbors. The experimental results show that the random removal technique produces good performance [36].

Gokulakrishnan *et al.* [38] applied an over-sampling method called Synthetic Minority Oversampling Technique (SMOTE) to tackle class imbalance in the case of sentiment analysis for Twitter data. Then, various methods were used including naïve Bayes, sequential mining optimization, random forest, and support vector machines. Gokulakrishnan *et al.* [38] showed that SMOTE was able to increase the accuracy for all methods. In [2], the authors used SMOTE, Borderline-SMOTE, and adaptive synthetic (ADASYN) to solve class imbalance and then used logistic regression and decision trees as sentiment classification methods. Similar to [38], Ah-Pine and Soriano-Morales [2] also reported that SMOTE produces better performance than other methods such as ADASYN.

Another technique used to redress class imbalance is ensemble learning. The ensemble framework applied a bootstrapping technique and used the same sample for both positive and negative classes [39]. To choose the best bootstrap model, the authors propose a step-wise iterative model selection. Similar to other research, Hassan *et al.* [39] employed traditional machine learning such as naïve Bayes, support vector machines, and logistic regression. Furthermore, Li *et al.* [40] proposed an active learning-based method called co-selecting to deal with class imbalance for sentiment analysis in product review datasets. This approach uses two feature subspace classifiers to identify the most informative samples minority-class.

Krawczyk *et al.* [41] proposed to address class imbalance by using a one-versus-one binary decomposition in a Twitter dataset. Then for each pairwise class, the dimensionality of feature space is reduced using multiple correspondence analysis. In these reduced dimensions, the procedure then applies three methods to fix the class imbalance, namely random undersampling, random oversampling, and SMOTE. The method then train a combination of binary and a weighted multi-class classifier. This combination gives more weight to the minority class. However, none of these works uses the class imbalance technique followed by deep learning techniques.

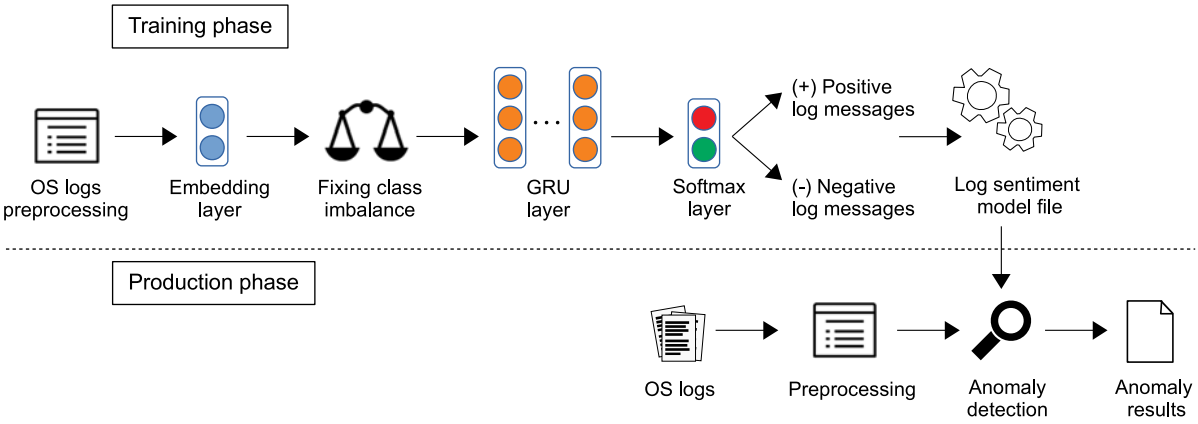


Fig. 2. The proposed method for sentiment analysis to detect anomalous log messages.

## 3 Threat Model and Assumptions

The log files save various activities on an OS. The interactions between the users and the system are recorded as well. For example, the system records error messages or warnings when an unauthorized user attempts to access a computer server, as illustrated in Fig. 1. The proposed *pylogsentiment* is not associated with a specific threat or attack to an OS. We identify a log entry as an anomaly when the message in the entry shows a negative sentiment.

The proposed anomaly detection method deals with point anomaly type, where each log entry is checked for its sentiment. The sentiment model for anomaly detection can be deployed in real-time or after a security incident has occurred. However, we do not provide real-time data training to update the model file. Moreover, if log entries do not provide human-readable message description, such as *[<c010ce54>] mtrr_wrmsr+0xf/0x2e* in a kernel log, we set them as a positive sentiment in the training phase.

## 4 The Proposed Method: *pylogsentiment*

An overall block diagram of *pylogsentiment* is shown in Fig. 2. There are two phases in our anomaly detection technique, namely training phase and production phase. In the training phase, we first preprocess the OS log files to extract log messages, which contain the sentiment. Second, we construct word embedding of log messages. Then we address the class imbalance issue with the Tomek link method before feeding the embedded vectors of log messages into the GRU layer. The last step in the training phase contains a softmax layer to decide whether a log message has either a negative or positive sentiment. Note that the negative polarity is related to anomalous activities, which may have occurred. We save the trained model in order to be deployed in the production server.

In the production phase, we also preprocess the raw OS logs before checking for anomalies. Detection is conducted based on the sentiment model that is constructed in the training phase. The system administrator then can check the anomaly results for further examination.

### 4.1 Preprocessing of Operating System Logs

A log entry has several entities such as timestamp, hostname, service name, and the main message. The entity which has sentiment is the message, so we need to extract it from the log entry. This extraction is illustrated in Fig. 3. Note that we focus on the message of a log entry as this part contains sentiment, which will be analyzed. Therefore, in the analysis, we do not consider other entities such as timestamp and hostname.

We use the nerlogparser [42] to parse system log files and extract the main message in each log entry. The nerlogparser tool is based on named entity recognition (NER), which is a process used to extract named entities from text. In log files, nerlogparser defines named entities as words or phrases containing common fields in a log entry such as timestamp, host name, or service name. The extraction of named entities is the equivalent of identifying each field in a log entry.

The nerlogparser utilizes bidirectional long short-term memory networks to perform NER [42]. The main benefit of the nerlogparser is that it provides automatic parsing because it is completed with a pre-trained model. Therefore, we do not need to define any rules or regular expressions. It can parse various log files as the nerlogparser has been trained in diverse types of logs. We define the parsed log messages, which contain sentiment as $\mathbf{M} = \{M_1, M_2, \ldots, M_{|\mathbf{M}|}\}$ and $M_i = \{w_{i1}, w_{i2}, \ldots, w_{i|M_i|}\}$ where $w_i$ is a single word in a message $M_i$, $|\mathbf{M}|$ is the total length of log messages, and $|M_i|$ is the length of a particular log message $M_i$.

### 4.2 Word Embedding as Input Layer

Before constructing the embedding layer, each word is converted into lowercase. Furthermore, the length of each log message is different. We make the size of the messages equal to the embedding size. If the log message is shorter than the embedding size, we pad it with zeros. On the other hand, if it is longer than the embedding size, we truncate it to the designated size. The padding and truncating is done to enable the data training to be run efficiently in batches because each batch has to be in the same size.

Before feeding into the neural networks, the first step in building a deep learning architecture is to convert raw data into vectors of numbers. First, we build a dictionary


Fig. 3. The preprocessing step to extract a log message containing sentiment.

containing a conversion of each word in all log messages to a unique integer. Let us denote the vocabulary size as $v$ The embedding layer will lookup each log message into a proper vector of integer. The procedure retrieves a sequence of integers for an input log message from a dictionary, which has been built previously. The dictionary is defined as $D \in \mathbb{Z}^v$. As discussed in the previous section, a log message $M_i$ is defined as:

$$M_i = \{w_{i1}, w_{i2}, \ldots, w_{i|M_i|}\},\ w \in D,\ i \in [0, |\mathbf{M}|]. \tag{1}$$

Let $k$ be the embedding size. To get embedded representation $M_i'$ for each log message $M_i$, we define

$$e_i = D(w_i),\ e_i \in \mathbb{Z}^v \tag{2}$$

$$M_i' = [e_1, e_2, \ldots, e_l],\ M_i' \in \mathbb{Z}^{|\mathbf{M}| \times k} \tag{3}$$

where $e_i$ is the word embedding of the $i$-th word in the log message $M_i$ and $D$ is a dictionary of vocabulary that has already been built. For each integer value, we look up its vector representation from a pre-trained embedding, namely GloVe which provides a embedding vector of each word based on statistical properties, such as the occurrence in a context or a sentence [43]. Note that $M_i'$ includes zero padding when $|M_i| < k$. The final embedding vector $\mathbf{E} \in \mathbb{Z}^{|\mathbf{M}| \times k}$ is then passed to the Tomek link method to fix the class imbalance problem.

### 4.3 Solving Class Imbalance With the Tomek Link

We need to address the class imbalance problem to obtain better deep learning model outputs. If we consider only imbalanced data, the model will tend to learn for the majority class only. For example, if positive data has 1000 data and negative has 100 data, the model can achieve 90 percent accuracy only by detecting the positive sentiment. On the other hand, in many applications especially in OS logs, the detection of negative sentiment is even more critical. It is important to address class imbalance because most real-life OS logs are unbalanced with negative log messages being in the minority.

We use the under-sampling method for class balancing. With balanced data, the model will be able to detect positive and negative sentiment proportionally. We use the Tomek link [12] to create a new representation of the majority class by under-sampling. We chose this method because it removes the samples from the majority class based on the minimum distance. In OS log data, the majority class is often repetitive and can be considered as a noise. Therefore, we can remove several log data from the majority class without affecting the performance.

We can view the Tomek link as a pair of nearest neighbors from different classes [44]. This means that this pair has the most minimum distance compared to other possible pairs. Let us define $S_{maj}$ as the majority class and $S_{min}$ as the minority class. Note that there are only two classes considered in this paper, specifically positive majority and negative minority. We then denote a pair of vectors $(\mathbf{x}_i, \mathbf{x}_j)$ from embedding vectors of log messages $\mathbf{E}$, where $\mathbf{x}_i \in S_{min}, \mathbf{x}_j \in S_{maj}$, and $d(\mathbf{x}_i, \mathbf{x}_j)$ is the euclidean distance between $\mathbf{x}_i$ and $\mathbf{x}_j$. $(\mathbf{x}_i, \mathbf{x}_j)$ is a Tomek link if there is no $\mathbf{x}_k$, such that:

$$d(\mathbf{x}_i, \mathbf{x}_k) < d(\mathbf{x}_i, \mathbf{x}_j) \text{ or} \tag{4}$$

$$d(\mathbf{x}_j, \mathbf{x}_k) < d(\mathbf{x}_i, \mathbf{x}_j). \tag{5}$$

If a pair of vectors $(\mathbf{x}_i, \mathbf{x}_j)$ form a Tomek link, then this pair is near to a class boundary. The procedure then removes the vector $\mathbf{x}_j$, which is a member of majority class $S_{maj}$. All Tomek links are removed until all nearest neighbor vector pairs are in the same class. The checking of whether or not a vector pair is in the same class uses the nearest neighbor method, where the number of neighbors is two.

In the Tomek link, the procedure only removes the *links* that are a pair of nearest neighbors between the majority and minority classes. Other undersampling methods delete many data from the majority class until the number of data between both classes is the same. In contrast, the Tomek link only removes the *links*. Consequently, the Tomek link does not remove many data instances from the majority class. By following this procedure, we obtain balanced data for training and validation. Subsequently, the GRU layer receives relatively balanced input data for sentiment classification, which is the balanced embedding vectors $\mathbf{E}'$.

### 4.4 Gated Recurrent Unit (GRU) Layer

GRU [11] can learn the long-term dependencies in a sequence and the positional relation of the whole log messages. GRU is an improvement of LSTM as it has only two gates in its cell, namely the reset gate and the update gate. The reset gate $r$ is calculated as:

$$r = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1}), \tag{6}$$

where $\sigma$ denotes the logistic sigmoid function, $\mathbf{x}$ is the input vector and $\mathbf{h}_{t-1}$ is the previous hidden state. Furthermore, $\mathbf{W}_r$ and $\mathbf{U}_r$ are weight matrices, which are processed to be optimized. Note that input $\mathbf{x}$ is the balanced embedding vectors $\mathbf{E}'$ in this case. The reset gate manages when the hidden state neglects the previous state and then adjusts with the current input. Moreover, the update gate $z$ is defined as:

$$z = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1}), \tag{7}$$

where the notation is similar with the reset gate $r$. The update gate manages information from the previous state to the current one [11].

The activation unit $h_t$ is denoted by:

$$h_t = z \odot h_{t-1} + (1 - z) \odot \tilde{h}_t, \tag{8}$$

where $\tilde{h}_t$ is calculated by:

$$\tilde{h}_t = \tanh(\mathbf{W}_{\tilde{h}_t} \mathbf{x}_t + \mathbf{U}_{\tilde{h}_t}(r \odot \mathbf{h}_{t-1})), \tag{9}$$

where $\odot$ is the element-wise product.

### 4.5 Softmax as Output Layer

To estimate the type of sentiment for each log message, *pylogsentiment* uses a softmax output layer. This layer predicts a normalized distribution over two possible labels for every log message. The softmax is given by:

$$s(\mathbf{x}_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}, \tag{10}$$

TABLE 1
A List of Public OS Logs Datasets Used in This Article

| Identifier | Description | # files | # lines | # positive | # negative |
|---|---|---|---|---|---|
| Casper | An ext3 disk image from Digital Corpora [45] | 15 | 11,086 | 9,874 | 1,212 |
| Jhuisi | The jhuisi host from DFRWS Challenge 2009 [46] | 25 | 11,737 | 9,063 | 2,674 |
| Nssal | The nssal host from DFRWS Challenge 2009 [46] | 40 | 107,093 | 91,349 | 15,732 |
| Honey7 | A Linux image from Honeynet Challenge 7 2011 [47] | 12 | 8,712 | 8,162 | 550 |
| Zookeeper | Logs from a distributed system deployed at CUHK [48] | 1 | 74,380 | 25,873 | 48,507 |
| Hadoop | Hadoop logs for big data processing [49] | 978 | 394,308 | 382,870 | 11,438 |
| BlueGene/L | Logs from a BlueGene/L super computer at LLNL [50] | 1 | 4,747,963 | 4,399,486 | 348,477 |
| Spark* | Spark logs for big data processing at CUHK [48] | 3852 | 33,236,604 | 31,513,147 | 1,723,457 |
| Honey5* | A Linux system from Honeynet Challenge 5 [51] | 7 | 124,386 | 67,798 | 56,588 |
| Windows* | Windows 7 logs from a CUHK laboratory [48] | 1 | 25,000 | 18599 | 6401 |

*Not included in the training phase. These datasets are for testing unknown anomalies in Section 5.6.*

where $\mathbf{x}_i = (x_1, x_2, \ldots x_n)$ is the output from the GRU layer. Furthermore, the objective of the proposed method as a supervised learning is to minimize the cross-entropy loss $H$, which is calculated by:

$$H(g, s) = -\sum_i g_i \; log(s_i), \tag{11}$$

where $g$ is the ground truth distribution and $s$ is the estimated distribution from the softmax function.

## 5 Experimental Results and Analysis

In this experiments, we first describe the OS log datasets and experiment settings. After that, we compare *pylogsentiment* with other anomaly detection methods and evaluate the Tomek link with other class balancing techniques. Subsequently, we compare *pylogsentiment* to other major deep learning techniques, such as convolutional neural networks (CNN), recurrent neural networks (RNN), long short-term memory (LSTM), and gated recurrent unit (GRU). These deep learning techniques have been used to address the sentiment analysis problem in other areas, such as social media or product review data. However, in our experiments, we test these architectures on OS log data. Finally, we discuss the limitations of the proposed method.

### 5.1 Operating System Log Datasets

To test the performance of the proposed method, we use four public datasets of OS logs and other system logs as shown in Table 1. These datasets include various attack scenarios, which lead to many anomalous or suspicious activities in log messages. This means that there are many log entries containing negative sentiments. We manually label all of the log messages from the datasets to obtain the ground truth. Table 1 also shows that there are class imbalances between positive and negative events in all OS log datasets. The first system logs are extracted from a disk image, namely nps-2009-casper-rw from Digital Corpora [45]. It is an ext3 file system dump from a bootable USB. This dataset provides OS logs from a Linux machine.

The second and third OS logs come from an annual security conference, namely The Digital Forensic Research Workshop (DFRWS). In 2009, they provided a challenge associated with OS log forensics. In the DFRWS Forensic Challenge 2009 [46], various log files had to be investigated in order to trace an attacker who illegally transferred secret data. There were two hosts involved in this case, namely "jhuisi" and "nssal". These two hosts were Sony PlayStation 3 (PS3) devices that run on a Linux OS. "nssal" is the biggest dataset with 91,349 positive and 15,732 negative entries.

Another OS logs are retrieved from Honeynet Forensic Challenge 7 2011 [47]. This dataset also provides a disk image of a compromised Linux server. From 12 files, there are 8,162 and 550 for positive and negative activities, respectively. For all datasets, we extracted the directory */var/log/* from the cloned disk images. We then acquired some common log files such as authentication logs, kernel logs, and system logs.

To test *pylogsentiment* against different types of domains, we also use other public system logs. Zookeeper is a management system for distributed systems. The logs were acquired from 32 hosts at the CUHK (Chinese University of Hong Kong) laboratory for 26 days [48]. Hadoop is a big data tool that can distribute jobs across machines. The logs were generated in a Hadoop cluster with 46 cores across five machines [49]. The anomalous events are machine down and network disconnections. BlueGene/L is an open dataset from a BlueGene/L supercomputer at Lawrence Livermore National Laboratories (LLNL) with 131,072 processors and 32,768 GB memory. The log entries include alert and non-alert logs. The alert messages indicate anomalous activities.

The last three datasets are not included in the training phase. These datasets are for testing unknown anomalies, as discussed in Section 5.6. Spark is a management tool for big data processing. The logs were collected from 32 hosts where include both normal and anomalous activities from the Spark system [48]. Honey5 is from The Forensic Challenge 5 2010, The Honeynet Project [51]. This dataset is a compromised Linux operating system. The directory */var/log/* has been imaged and the system analyst needs to analyze the brute-force attack recorded in the log files. Finally, Windows logs were collected from a Windows 7 machine at CUHK laboratory [48]. The Windows had a CBS (Component Based Servicing) configured for a secure and controlled software installation.

### 5.2 Experiment Settings

We implement *pylogsentiment* using Python 3.5 and Keras 2.1.5 [52] with TensorFlow 1.8 [53] as the backend. We use imbalanced-learn library [54] to implement the Tomek link and compare it with various class balancing methods. To

TABLE 2
Performance Comparison (%) of *pylogsentiment* With Other Anomaly Detection Techniques on the Casper Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Decision tree | 83.466 | 77.037 | 79.488 | 94.998 |
| Logistic regression | 66.959 | 60.863 | 59.700 | 90.993 |
| SVM | 58.614 | 60.757 | 59.482 | 90.967 |
| DeepLog | 98.457 | 89.275 | 93.246 | 97.612 |
| Isolation forest | 52.407 | 50.650 | 49.926 | 88.149 |
| PCA | 51.205 | 50.282 | 49.362 | 87.480 |
| *pylogsentiment* | **99.487** | **99.413** | **99.449** | **99.459** |

TABLE 3
Performance Comparison (%) of *pylogsentiment* With Other Anomaly Detection Techniques on the Jhuisi Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Decision tree | 91.534 | 89.769 | 90.550 | 93.313 |
| Logistic regression | 68.373 | 66.127 | 64.886 | 79.182 |
| SVM | 63.643 | 66.506 | 64.051 | 80.914 |
| DeepLog | 98.405 | 96.981 | 97.700 | 98.383 |
| Isolation forest | 57.519 | 52.774 | 51.700 | 74.707 |
| PCA | 44.828 | 49.299 | 45.014 | 75.448 |
| *pylogsentiment* | **98.867** | **98.761** | **98.813** | **98.850** |

run the experiments for the proposed and baseline methods, we use a computer with 12 cores of CPU, 64 GB of RAM, and NVIDIA GeForce GTX 1080 Ti 12 GB GPU. The output of training and validation step is a model file, which is used for the testing step and in production to detect sentiment in log entries.

Hyperparameters used in the proposed method are explained as follows. The maximum epoch for training is 50 with a dropout rate of 0.4. We use the batch size of 128 and early stopping of five times. This means that the training will stop if there is no more improvement after five epochs. We use Adam [55] as the learning method in GRU architecture. Adam is good for general cases because it is efficient, it needs only a small amount of RAM, and it is invariant to gradient scaling. We set the learning rate to 0.001. We set the batch size to 64 and GRU hidden states are set to 64. We split all datasets according to these proportions: 60 percent for training, 20 percent for development, and 20 percent for testing. We merge datasets for training and validation, respectively. Therefore, there is only one deep learning model that can be tested to each testing dataset.

For evaluation metrics, we use precision, recall, F1, and accuracy. Precision is the ratio of the number of true positives and the sum of true and false positives. Recall focuses on measuring the ability of the methods to detect positive log messages. We can calculate F1 from precision and recall and it is calculated by:

$$\text{precision} = \frac{TP}{TP + FP} \tag{12}$$

$$\text{recall} = \frac{TP}{TP + FN}, \tag{13}$$

where $TP$ is true positives, $FP$ is false positives, and $FN$ is false negatives. Furthermore, the F1 is defined as:

$$\text{F1} = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}. \tag{14}$$

For each dataset, we calculate the F1 for each file and then obtain the mean F1 for all log files.

Finally, accuracy provides a proportion of true results for both positive and negative sentiments among the total number of log messages in the experiment. Therefore, these four metrics can capture the performances of all methods in various aspects. For each dataset, we calculate all evaluation metrics and then obtain the mean for all datasets. Precision, recall, and F1 are implemented with the scikit-learn library [56] with the "macro average" option, which means averaging the unweighted mean per label.

### 5.3 Comparison With Other Log Anomaly Detection Methods

We compared *pylogsentiment* with other log anomaly detection techniques from the loglizer toolbox[2] [14]. They are four supervised detection methods, namely decision tree [17], logistic regression [18], and support vector machines (SVM) [19], and DeepLog [21]. In addition, we perform a comparison with two unsupervised detection methods, namely isolation forest [15] and principal component analysis (PCA) [16]. There are other methods provided in the loglizer tool, such as log clustering and invariants mining. However, we chose these methods because they provided better performance than others.

Before feeding the log files to the loglizer, we split all log entries to obtain the main message using the nerlogparser tool [42]. After that, we extract event sequences using the Drain method [57]. These event sequences are then processed by the loglizer. For large-scale log files, one can use a parallel log parser, namely POP [58] to extract events before running the loglizer tool. The performance comparison of *pylogsentiment* and the five other anomaly detectors are reported in Table 2 to Table 8. Note that the bold values in all tables indicate the best performance.

Among four supervised methods from the loglizer, the DeepLog shows the best performance because it uses deep learning to detect anomalies. The second best from the loglizer is the decision tree method. For instance, it achieved an F1 score of 90.550, 89.470, and 86.359 percent, respectively, on the Jhuisi (Table 3), the Nssal (Table 4), and the Honey7 dataset (Table 5). Unlike logistic regression that only works best in linear classification, the performance of the decision tree is not affected by the linearity. The decision tree is a straightforward approach for binary classification, such as sentiment analysis. Therefore, the decision tree provides the good performance compared to other methods from loglizer toolbox.

The performance of logistic regression on all datasets are similar to the SVM. For example, they provide similar accuracy of 98.562 and 98.078 percent, respectively, on the Zookeper dataset (Table 6). A difficulty of SVM to reach optimal performance is that we have to tune the hyperparameters used in the training step carefully. In these experiments, we

2. https://github.com/logpai/loglizer

TABLE 4
Performance Comparison (%) of *pylogsentiment* With Other Anomaly Detection Techniques on the Nssal Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Decision tree | 94.791 | 87.700 | 89.470 | 98.063 |
| Logistic regression | 85.133 | 74.728 | 76.476 | 97.604 |
| SVM | 80.206 | 74.935 | 76.474 | 97.655 |
| DeepLog | 95.245 | 90.258 | 92.535 | 96.461 |
| Isolation forest | 65.504 | 57.352 | 56.101 | 80.967 |
| PCA | 52.642 | 53.827 | 49.505 | 80.614 |
| *pylogsentiment* | **97.170** | **96.050** | **96.602** | **99.020** |

TABLE 5
Performance Comparison (%) of *pylogsentiment* With Other Anomaly Detection Techniques on the Honey7 Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Decision tree | 93.260 | 83.307 | 86.359 | 97.471 |
| Logistic regression | 68.143 | 70.000 | 69.042 | 96.286 |
| SVM | 68.143 | 70.000 | 69.042 | 96.286 |
| DeepLog | 96.864 | 95.271 | 96.052 | 99.082 |
| Isolation forest | 47.509 | 50.948 | 48.064 | 85.966 |
| PCA | 60.871 | 58.898 | 55.943 | 88.279 |
| *pylogsentiment* | **99.970** | **99.107** | **99.535** | **99.943** |

TABLE 6
Performance Comparison (%) of *pylogsentiment* With Other Anomaly Detection Techniques on the Zookeeper Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Decision tree | 98.599 | 98.880 | 98.737 | 98.851 |
| Logistic regression | 98.369 | 98.464 | 98.416 | 98.562 |
| SVM | 97.987 | 97.769 | 97.877 | 98.078 |
| DeepLog | 98.851 | 99.353 | 99.094 | 99.173 |
| Isolation forest | 32.607 | 50.000 | 39.473 | 65.215 |
| PCA | 79.011 | 51.209 | 42.121 | 66.021 |
| *pylogsentiment* | **99.722** | **99.898** | **99.810** | **99.973** |

TABLE 7
Performance Comparison (%) of *pylogsentiment* With Other Anomaly Detection Techniques on the Hadoop Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Decision tree | 48.523 | 50.000 | 49.250 | 97.046 |
| Logistic regression | 48.523 | 50.000 | 49.250 | 97.046 |
| SVM | 48.523 | 50.000 | 49.250 | 97.046 |
| DeepLog | 88.256 | 98.352 | 92.700 | 99.062 |
| Isolation forest | 47.702 | 50.000 | 48.824 | 54.034 |
| PCA | 49.995 | 49.996 | 49.996 | 58.214 |
| *pylogsentiment* | **99.886** | **99.732** | **99.809** | **99.905** |

tune the parameter of the traditional machine learning methods using the grid search technique. It is an exhaustive search over predefined parameter values for the classifiers. Most of the methods in the loglizer are from scikit-learn library. Therefore, we also implement the grid search techniques in scikit-learn.

The isolation forest and PCA algorithm broadly failed to identify anomalous events in repeated log messages because they have a very similar pattern. It is indicated by low precision and recall values on all datasets. For example in Table 7, the isolation forest achieved only 47.702 percent precision, while the PCA showed 49.995 percent precision. In general, these unsupervised methods show lower performances compare to supervised ones. The results are clear because the unsupervised techniques do not learn the OS logs data and heavily rely on the anomaly threshold given by the user.

The overall performances of methods from the loglizer tool are lower than *pylogsentiment* because they use count matrix as the features and sequence of events as the inputs, specifically for the DeepLog. These features then inputted into various machine learning in the loglizer.

The count matrix has several disadvantages. First, it only considers the frequency of events as features. One can add more features such as the inter-arrival rate of events to provide more information to the classifier about anomalous events. Second, the count matrix is grouped based on sequence windows. In the case of OS logs, the anomalies do not always appear in each window. This situation also influences the training process by machine learning methods in the loglizer tool. This windowing mechanism also does not fit well with the point anomaly type. Such technique will be a good fit for collective anomalies.

In line with the results from He et al. [14], the supervised methods provide better performance compared to the unsupervised ones. However, *pylogsentiment* still demonstrates a superior performance with overall mean F1 and accuracy of 99.135 and 99.590 percent, respectively. The reason is that *pylogsentiment* checks for each content of log messages rather than extracting only the frequency information from the OS logs.

The overall performance of *pylogsentiment* and other anomaly detection methods. We present all evaluation metrics and the value is calculated by obtaining the mean value for all datasets. Based on Table 2 to Table 8, we can see that the proposed method delivers superior performance on average than the other methods. This improvement is very important because the detection of negative sentiment in OS logs needs to be accurate so that the administrator can effectively utilize this sentiment-based approach.

### 5.4 Comparison of the Tomek Link With Other Class Balancing Methods

We apply the class balancing to all datasets using the Tomek link method. Fig. 4 depicts the performance of the Tomek link in comparison with other class balancing methods, namely ADASYN [59], SMOTE [60], random under sampler, instance hardness threshold [61], and neighborhood cleaning rule [62]. Note that these class balancing methods are positioned before the GRU layer and we save all metrics values after we run them on all datasets. In addition, we test weighted cross-entropy loss and focal loss [63] that are commonly used in deep learning to address class imbalance problem. The loss function is calculated in each epoch of training phase.

Besides the accuracy, we also consider other metrics specifically precision, recall, and F1 to check model performance against both classes. The reason is that the accuracy value may be misleading to provide a performance metric as discussed in Section 4.3.

TABLE 8
Performance Comparison (%) of *pylogsentiment* With Other Anomaly Detection Techniques on the BlueGene/L Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Decision tree | 60.576 | 50.998 | 50.303 | 92.348 |
| Logistic regression | 54.028 | 51.852 | 52.092 | 90.368 |
| SVM | 46.314 | 50.000 | 48.087 | 92.628 |
| DeepLog | 99.281 | 99.800 | 99.539 | 99.879 |
| Isolation forest | 53.081 | 50.047 | 51.519 | 47.389 |
| PCA | 51.168 | 54.260 | 38.970 | 48.487 |
| *pylogsentiment* | **99.892** | **99.963** | **99.928** | **99.980** |

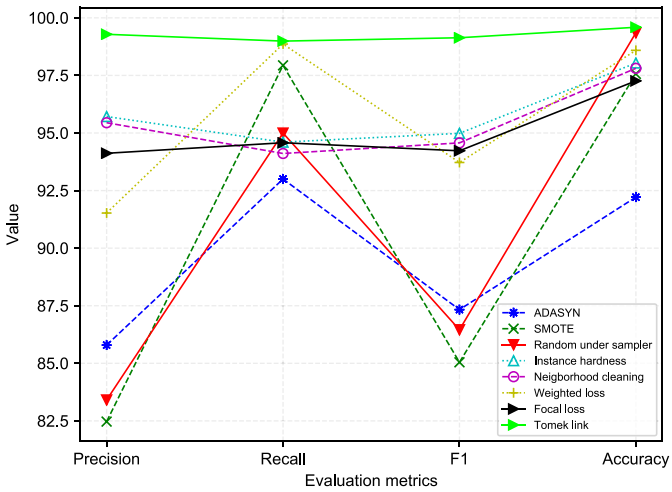


Fig. 4. Comparison of various data balancing methods when combined with the GRU.

As shown in Fig. 4, the Tomek link showed the best overall value for precision, recall, F1, and accuracy with 99.836, 99.839, 99.837, and 99.931 percent, respectively. The recall value of the Tomek link with 99.839 percent is slightly better than the weighted cross-entropy loss with 98.863 percent. However, the Tomek link is superior in other three metrics. On the other hand, SMOTE has same performance with the instance hardness threshold in our case. In the four datasets used in the experiment, the majority class is positive. As we use the under-sampling method, the size of positive class as the majority is reduced. Note that this sampling still produces good performance because many positive log entries are very similar to each other.

### 5.5 Comparison With Other Deep Learning-Based Sentiment Analysis Methods

The proposed method, *pylogsentiment*, uses the GRU technique on top of the Tomek link method for class balancing. To show that the proposed method can achieve the best performance, we compare the proposed method with five baseline methods as listed below.

1) *CNN*. CNN has been implemented for sentence classification including sentiment analysis [24].
2) *RNN*, *LSTM*, and *GRU*. Three major deep learning architectures, specifically RNN, LSTM, and GRU are also applied to all datasets.
3) *BRNN*. Bidirectional RNN from [64]. Bidirectional technique considers information from forward and backward direction in the word sequences of the log text data.

TABLE 9
Performance Comparison (%) of *pylogsentiment* With Other Deep Learning Techniques on the Casper Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| CNN | 98.959 | 91.358 | 94.744 | 98.107 |
| RNN | 86.587 | 54.014 | 54.769 | 89.770 |
| BRNN | 90.687 | 70.839 | 76.723 | 93.060 |
| LSTM | 92.728 | 90.238 | 91.433 | 96.755 |
| GRU | 93.614 | 90.751 | 92.117 | 97.026 |
| *pylogsentiment* | **99.975** | **99.794** | **99.884** | **99.955** |

TABLE 10
Performance Comparison (%) of *pylogsentiment* With Other Deep Learning Techniques on the Jhuisi Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| CNN | 94.838 | 90.290 | 92.309 | 94.851 |
| RNN | 90.319 | 87.508 | 88.801 | 92.383 |
| BRNN | 94.832 | 92.402 | 93.544 | 95.574 |
| LSTM | 91.660 | 90.773 | 91.206 | 93.872 |
| GRU | 97.976 | 98.039 | 98.007 | 98.596 |
| *pylogsentiment* | **99.769** | **99.506** | **99.637** | **99.745** |

We compare all baseline methods and *pylogsentiment* on seven datasets as shown in Table 9 to Table 15. The results of the comparison for the Casper dataset are shown in Table 9 where the bold value indicates the best performance. *pylogsentiment* gives the best performance with sophisticated values for all metrics as follows: 99.975 percent precision, 99.794 percent recall, 99.884 percent F1 score, and 99.955 percent accuracy.

The experimental results on the Jhuisi dataset for all baseline methods and *pylogsentiment* are shown in Table 10. The proposed method which uses GRU and the Tomek link outperforms other methods. With fewer gates than LSTM, *pylogsentiment* still generalizes well on Jhuisi dataset. The performance is also better than its closest competitor, specifically GRU, on this dataset with 99.637 and 99.745 percent for F1 and accuracy, respectively.

In all datasets, the performance of regular GRU is comparable to the proposed method as indicated in Table 11. Furthermore, adding the class balance method to GRU improves the sentiment detection performance. On the Honey7 dataset, *pylogsentiment* still offers superior performance than other methods as shown in Table 12 with F1 score of 99.535 percent. In addition, it shows that the sequence model such as LSTM and GRU are more effective than using CNN for detecting sentiment in several cases of system logs.

The use of Tomek link as an under-sampling method also provides a better model as it learns from balanced data, while other methods are applied to imbalanced data. RNN as the base method for LSTM and GRU showed poor results for all datasets. This is why LSTM and GRU come to improve the RNN. On the Zookeeper and Hadoop datasets, GRU is better than LSTM with accuracy of 99.140 percent (Table 13) and 99.930 percent (Table 14), respectively. It is broadly expected because GRU is an improvement of LSTM with fewer gates inside its architecture.

TABLE 11
Performance Comparison (%) of *pylogsentiment* With Other Deep Learning Techniques on the Nssal Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| CNN | 98.144 | 97.802 | 97.972 | 98.987 |
| RNN | 93.582 | 96.334 | 94.892 | 97.357 |
| BRNN | 97.797 | 97.847 | 97.822 | 98.907 |
| LSTM | 92.900 | 97.005 | 94.804 | 97.269 |
| GRU | 98.700 | 99.326 | 99.010 | 99.500 |
| *pylogsentiment* | **99.835** | **99.848** | **99.842** | **99.921** |

TABLE 12
Performance Comparison (%) of *pylogsentiment* With Other Deep Learning Techniques on the Honey7 Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| CNN | 94.063 | 88.785 | 91.229 | 98.049 |
| RNN | 80.029 | 70.113 | 73.913 | 94.836 |
| BRNN | 89.833 | 88.479 | 89.142 | 97.476 |
| LSTM | 96.245 | 97.028 | 96.632 | 99.197 |
| GRU | 99.410 | 98.151 | 98.771 | 99.713 |
| *pylogsentiment* | **99.550** | **99.969** | **99.758** | **99.943** |

TABLE 13
Performance Comparison (%) of *pylogsentiment* With Other Deep Learning Techniques on the Zookeeper Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| CNN | 98.691 | 99.124 | 98.901 | 98.999 |
| RNN | 98.741 | 99.225 | 98.976 | 99.066 |
| BRNN | 98.623 | 98.853 | 98.736 | 98.851 |
| LSTM | 98.776 | 99.232 | 98.997 | 99.086 |
| GRU | 98.839 | 99.286 | 99.056 | 99.140 |
| *pylogsentiment* | **99.990** | **99.995** | **99.993** | **99.993** |

TABLE 14
Performance Comparison (%) of *pylogsentiment* With Other Deep Learning Techniques on the Hadoop Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| CNN | 99.608 | 98.788 | 99.195 | 99.910 |
| RNN | 96.332 | 98.684 | 97.477 | 99.708 |
| BRNN | 99.675 | 98.790 | 99.228 | 99.914 |
| LSTM | 98.684 | 98.869 | 98.776 | 99.861 |
| GRU | 99.770 | 98.989 | 99.376 | 99.930 |
| *pylogsentiment* | **99.841** | **99.799** | **99.820** | **99.980** |

TABLE 15
Performance Comparison (%) of *pylogsentiment* With Other Deep Learning Techniques on the BlueGene/L Dataset

| Method | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| CNN | 99.878 | 99.477 | 99.676 | 99.904 |
| RNN | 99.394 | 99.318 | 99.356 | 99.825 |
| BRNN | 98.789 | 99.058 | 98.923 | 99.714 |
| LSTM | 99.281 | 99.800 | 99.539 | 99.879 |
| GRU | 98.704 | 99.460 | 99.078 | 99.726 |
| *pylogsentiment* | **99.892** | **99.963** | **99.928** | **99.980** |

TABLE 16
Comparison of Detected Anomalies by GRU and *pylogsentiment*

| Dataset | Total anomalies on testing dataset | Detected anomalies | |
|---|---|---|---|
| | | GRU only | *pylogsentiment* |
| Casper | 243 | 201 | **242** |
| Jhuisi | 536 | 520 | **531** |
| Nssal | 3147 | 3118 | **3139** |
| Zookeeper | 110 | 106 | **110** |
| Hadoop | 9702 | 9586 | **9701** |

In non-OS log datasets (Table 13 to Table 15), *pylogsentiment* still offers the best performance because it is a domain-agnostic method, which means it can be applied to any human-readable and text-based system logs. For example, it achieved 99.993 and 99.980 percent accuracy on the Zookeeper and Hadoop dataset as displayed in Table 13 and Table 14, respectively. Based on the experimental results on non-OS log datasets, regular GRU provide slightly weaker of overall performance compared to *pylogsentiment*.

On the other hand, the worst performance is demonstrated by regular RNN on OS log datasets (Table 9 to Table 12). The reason is that RNN cannot remember the long-term dependencies between words in log messages. BRNN provides performance improvement to RNN as the bidirectional technique train the word vector sequences in both forward and backward directions.

We have demonstrated that the use of GRU in *pylogsentiment* produces better results than LSTM in the case of anomaly detection in system logs. As shown in Table 9 to Table 15, the performance of *pylogsentiment*, which uses Tomek link and GRU outperforms its LSTM based counterpart. The main reason is that log data comprises of short and simple text. Therefore, it does not need any strong and long dependency features provided by LSTM. GRU with a fewer gates is more suitable with the characteristics of the log message data.

Furthermore, while a bulk of the improvement in anomaly detection performance has been achieved by the deep learning architecture (i.e., GRU), the Tomek link as a class balancing technique, further improves the performance. To demonstrate the importance of the Tomek link, an ablative analysis is shown in Table 16. As shown in Table 16, for example, for Casper dataset, GRU alone can detect 201 anomalies from the total of 243 anomalies in the test dataset. Our proposed *pylogsentiment*, which is a combination of GRU and Tomek link, was able to detect 242 anomalies. It means, *pylogsentiment* detected 41 additional anomalies out of the remaining 42. A similar trend can be seen for other datasets. This additional improvement is significant because anomalous log entries are rare and important.

In summary, we can state that the proposed method outperforms all other methods in terms of overall precision, recall, F1, and accuracy with 99.836, 99.839, 99.837, and 99.931 percent, respectively. The reason is that the GRU which provides better composition capability than other sequence models, such as RNN or its variants. As GRU has fewer gates than LSTM, it also can generalize better than LSTM in the case of OS log data. Moreover, the class balancing achieved with the

TABLE 17
Performance of *pylogsentiment* (%) on Untrained Datasets

| Dataset | Precision | Recall | F1 | Accuracy |
|---|---|---|---|---|
| Spark | 99.451 | 92.447 | 95.651 | 99.205 |
| Honey5 | 99.448 | 99.487 | 99.467 | 99.471 |
| Windows | 83.884 | 91.119 | 85.992 | 88.028 |

Tomek link method also offers a better representation of raw log messages.

### 5.6 Detecting Anomalies on Unseen Datasets

We discuss the robustness or generalizability of *pylogsentiment* in detecting unknown anomalies on datasets that have not been trained before. These datasets include Spark logs [48], Honey5 (a compromised Linux host) logs [51], and Windows logs [48]. Note that *pylogsentiment* has never seen these datasets before. Nevertheless, *pylogsentiment* still provides a good performance to detect unknown anomalies as provided in Table 17. For instance, it achieved 95.651 and 99.467 percent F1 scores on the Spark and Honey5 datasets, respectively.

*pylogsentiment* has been trained on a number of training datasets (Table 1) and is able to deal with unknown anomalies as shown in Table 17. It is also applicable in a production environment as shown in Honey5 logs from a different Linux host. It also works well even on a significantly different type of unseen logs, specifically on Windows logs dataset, and it achieved a recall rate of 91.119 percent. The main reasons are two fold. First, we use the GloVe word embeddings [43] to represent log messages. GloVe is able to capture the relationship and meaning between words. Second, we use a supervised deep learning model combined with an under-sampling technique that achieved high performances on both previously seen and unseen datasets.

The example of the applicability of GloVe in contributing to detect unknown anomalies is as follows. Let us take one word "failed" that has close meaning to "error". In this example, let us assume "failed" is never seen in the training dataset, but "error" appears in training. We can still recognize a log entry containing the word "failed" as anomalous because they have close meaning based on GloVe word vectors and very high similarity of meaning with the vocabularies of log messages that have been trained.

In a production environment, we can easily deploy *pylogsentiment* by invoking this command after installation process as explained on the GitHub page[3]: `pylogsentiment -i log_file`, where `log_file` is an arbitrary log file to be checked for its anomalies. Note that there is only one final model file of *pylogsentiment* that can be applied to arbitrary log files. For the aforementioned reasons and an ease of deployment, the proposed method is applicable in detecting unknown anomalies and can be implemented on a production server.

### 5.7 Limitations

Despite its high accuracy to detect anomalous OS log messages, *pylogsentiment* has several limitations as discussed below.

*Training for More OS Log Datasets*. Deep learning models are highly dependent on the datasets to be learned from in the training phase. Public log datasets, such as Loghub,[4] can be used to make the sentiment model to generalize well across various events in OS logs or other application logs. From LogHub, we have already used three datasets for training and two other untrained datasets to test *pylogsentiment*. However, there are still more datasets to be included in the training phase.

*Mechanism for Model Update*. At the moment, *pylogsentiment* does not have a mechanism to update the sentiment model. This update is useful because the OS events or activities recorded in the log files will continue to change dynamically. The main source of the update procedure is the anomalous log inputs from the system administrators. The model update will improve the detection performance of the proposed method.

*Nonhuman-Readable OS Log Entries*. *pylogsentiment* only deals with human-readable log messages because it detects anomalies by identifying their sentiments. However, in several cases, there are log entries that are not human-readable. In this work, *pylogsentiment* still recognizes them as positive sentiments. A possible solution for this issue is to build an advanced deep learning method to recognize special entities such as kernel-related activity (e.g., *[<c010ce54>] mtrr_wrmsr+0xf/0x2e*, memory address (e.g., *lowmem : 0xc0000000 - 0xcfff0000 (255 MB)*), or another hardware-related log entry (e.g., *ACPI: RSDP 000E0000, 0024 (r2 VBOX)*). Subsequently, the deep learning model will determine these entities as anomalous or not.

## 6 Conclusion and Future Work

In this paper, we propose to apply sentiment analysis to detect anomalous activities in OS logs. We use a deep learning technique, namely gated recurrent unit (GRU) on top of the Tomek link as a class imbalance method. We consider class imbalance because, in real-life OS logs, the number of negative sentiments is smaller than the positive ones. To produce a better deep learning model, we first address this class imbalance issue. Based on the experimental results, the proposed method achieved the best performance compared to other baseline methods with F1 of 99.84 percent and accuracy of 99.93 percent. *pylogsentiment* is also applicable to other types of system logs.

In future, we will focus on the training of the proposed deep learning technique with other types of log datasets, such as application logs, to achieve generalization across log datasets. Therefore, the generated model can be readily used to detect anomalies in more various environments. *pylogsentiment* is trained for point anomaly detection, it can be extended to identify collective anomalies where the input is a sequence of log entries. In addition, we plan to incorporate the sentiment-based method to statistics-based anomaly detection [65]. This will lead to a greater accuracy in the detection of irregularities in OS logs. The combination of sentiment-based and statistics-based models will assist the system administrator to better monitor the OS logs.

3. https://github.com/studiawan/pylogsentiment
4. https://github.com/logpai/loghub

## ACKNOWLEDGMENTS

This work was supported by the Indonesia Lecturer Scholarship (BUDI) from Indonesia Endowment Fund for Education (LPDP), Ministry of Finance, Republic of Indonesia. The authors would like to acknowledge NVIDIA for providing a GPU for the experiments involved in this research. They also thank three anonymous reviewers who have provided constructive comments to this manuscript.

## REFERENCES

[1] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment classification using machine learning techniques," in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2002, pp. 79–86.

[2] J. Ah-Pine and E.-P. Soriano-Morales, "A study of synthetic oversampling for Twitter imbalanced sentiment analysis," in *Proc. Workshop Interact. Between Data Mining Natural Lang. Process.*, 2016, pp. 17–24.

[3] O. Araque, I. Corcuera-Platas, J. F. Sanchez-Rada, and C. A. Iglesias, "Enhancing deep learning sentiment analysis with ensemble techniques in social applications," *Expert Syst. Appl.*, vol. 77, pp. 236–246, 2017.

[4] H. Chen, M. Sun, C. Tu, Y. Lin, and Z. Liu, "Neural sentiment classification with user and product attention," in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2016, pp. 1650–1659.

[5] H. Studiawan, F. Sohel, and C. Payne, "A survey on forensic investigation of operating system logs," *Digit. Invest.*, vol. 29, pp. 1–20, 2019.

[6] D. Basin, P. Schaller, and M. Schläpfer, "Logging and log analysis," in *Applied Information Security*. Berlin, Germany: Springer, 2011, pp. 69–80.

[7] Splunk Inc., "Splunk tool," 2019. [Online]. Available: https://www.splunk.com/

[8] A. Bovenzi, F. Brancati, S. Russo, and A. Bondavalli, "An OS-level framework for anomaly detection in complex software systems," *IEEE Trans. Dependable Secure Comput.*, vol. 12, no. 3, pp. 366–372, May/Jun. 2014.

[9] T. Schindler, "Anomaly detection in log data using graph databases and machine learning to defend advanced persistent threats," in *Proc. Lecturer Notes Informat.*, 2017, pp. 2371–2378.

[10] L. Zhang, S. Wang, and B. Liu, "Deep learning for sentiment analysis: A survey," *Wiley Interdisciplinary Rev., Data Mining Knowl. Discov.*, vol. 8, no. 4, 2018, Art. no. e1253.

[11] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2014, pp. 1724–1734.

[12] I. Tomek, "Two modifications of CNN," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-6, no. 11, pp. 769–772, Nov. 1976.

[13] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, 2009, Art. no. 15.

[14] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *Proc. IEEE 27th Int. Symp. Softw. Rel. Eng.*, 2016, pp. 207–218.

[15] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.

[16] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM SIGOPS 22nd Symp. Operating Syst. Princ.*, 2009, pp. 117–132.

[17] M. Chen, A. X. Zheng, J. Lloyd, M. I. Jordan, and E. Brewer, "Failure diagnosis using decision trees," in *Proc. Int. Conf. Auton. Comput.*, 2004, pp. 36–43.

[18] P. Bodik, M. Goldszmidt, A. Fox, D. B. Woodard, and H. Andersen, "Fingerprinting the datacenter: Automated classification of performance crises," in *Proc. 5th Eur. Conf. Comput. Syst.*, 2010, pp. 111–124.

[19] Y. Liang, Y. Zhang, H. Xiong, and R. Sahoo, "Failure prediction in IBM BlueGene/L event logs," in *Proc. 7th IEEE Int. Conf. Data Mining*, 2007, pp. 583–588.

[20] R. Bitton and A. Shabtai, "A machine learning-based intrusion detection system for securing remote desktop connections to electronic flight bag servers," *IEEE Trans. Dependable Secure Comput.*, to be published, doi: 10.1109/TDSC.2019.2914035.

[21] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security.*, 2017, pp. 1285–1298.

[22] R. Vinayakumar, K. Soman, and P. Poornachandran, "Long short-term memory based operation log anomaly detection," in *Proc. Int. Conf. Adv. Comput., Commun. Informat.*, 2017, pp. 236–242.

[23] A. Brown, A. Tuor, B. Hutchinson, and N. Nichols, "Recurrent neural network attention mechanisms for interpretable system log anomaly detection," in *Proc. 1st Workshop Mach. Learn. Comput. Syst.*, 2018, pp. 1–8.

[24] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. 2014 Conf. Empir. Methods Nat. Lang. Process.*, 2014, pp. 1746–1751.

[25] X. Wang, Y. Liu, S. Chengjie, B. Wang, and X. Wang, "Predicting polarities of tweets by composing word embeddings with long short-term memory," in *Proc. 53rd Annu. Meet. Assoc. Comput. Linguistics*, 2015, pp. 1343–1353.

[26] H. Zhou, M. Huang, T. Zhang, X. Zhu, and B. Liu, "Emotional chatting machine: Emotional conversation generation with internal and external memory," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 730–738.

[27] J. L. Elman, "Finding structure in time," *Cogn. Sci.*, vol. 14, no. 2, pp. 179–211, 1990.

[28] C. Dos Santos and M. Gatti, "Deep convolutional neural networks for sentiment analysis of short texts," in *Proc. 25th Int. Conf. Comput. Linguistics: Tech. Papers*, 2014, pp. 69–78.

[29] A. Severyn and A. Moschitti, "Twitter sentiment analysis with deep convolutional neural networks," in *Proc. 38th Int. ACM SIGIR Conf. Res. and Develop. Inf. Retrieval*, 2015, pp. 959–962.

[30] R. Socher *et al.*, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2013, pp. 1631–1642.

[31] A. Radford, R. Jozefowicz, and I. Sutskever, "Learning to generate reviews and discovering sentiment," 2017, *arXiv : 1704.01444*.

[32] I. Weber, V. R. K. Garimella, and E. Borra, "Mining web query logs to analyze political issues," in *Proc. 3rd Annu. ACM Web Sci. Conf.*, 2012, pp. 330–339.

[33] E. Guzman, D. Azócar, and Y. Li, "Sentiment analysis of commit comments in GitHub: An empirical study," in *Proc. 11th Work. Conf. Mining Softw. Repositories*, 2014, pp. 352–355.

[34] M. Thelwall, K. Buckley, G. Paltoglou, D. Cai, and A. Kappas, "Sentiment strength detection in short informal text," *J. Amer. Soc. Inf. Sci. Technol.*, vol. 61, no. 12, pp. 2544–2558, 2010.

[35] V. Sinha, A. Lazar, and B. Sharif, "Analyzing developer sentiment in commit logs," in *Proc. 13th Int. Workshop Mining Softw. Repositories*, 2016, pp. 520–523.

[36] A. Mountassir, H. Benbrahim, and I. Berrada, "Addressing the problem of unbalanced data sets in sentiment analysis," in *Proc. Int. Conf. Knowl. Discov. Inf. Retrieval*, 2012, pp. 306–311.

[37] A. Mountassir, H. Benbrahim, and I. Berrada, "An empirical study to address the problem of unbalanced data sets in sentiment classification," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, 2012, pp. 3298–3303.

[38] B. Gokulakrishnan, P. Priyanthan, T. Ragavan, N. Prasath, and A. Perera, "Opinion mining and sentiment analysis on a Twitter data stream," in *Proc. Int. Conf. Adv. ICT Emerg. Regions*, 2012, pp. 182–188.

[39] A. Hassan, A. Abbasi, and D. Zeng, "Twitter sentiment analysis: A bootstrap ensemble framework," in *Proc. Int. Conf. Soc. Comput.*, 2013, pp. 357–364.

[40] S. Li, S. Ju, G. Zhou, and X. Li, "Active learning for imbalanced sentiment classification," in *Proc. Joint Conf. Empir. Methods Natural Lang. Process. Comput. Nat. Lang. Learn.*, 2012, pp. 139–148.

[41] B. Krawczyk, B. T. McInnes, and A. Cano, "Sentiment classification from multi-class imbalanced twitter data using binarization," in *Proc. Int. Conf. Hybrid Artif. Intell. Syst.*, 2017, pp. 26–37.

[42] H. Studiawan, F. Sohel, and C. Payne, "Automatic log parser to support forensic analysis," in *Proc. 16th Australian Digit. Forensics Conf.*, 2018, pp. 1–10.

[43] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 1532–1543.

[44] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[45] S. Garfinkel, "nps-2009-casper-rw: An ext3 file system from a bootable USB," 2009. [Online]. Available: http://downloads.digitalcorpora.org/corpora/drives/nps-2009-casper-rw/
[46] E. Casey and G. G. Richard III, "DFRWS Forensic Challenge 2009," 2009. [Online]. Available: http://old.dfrws.org/2009/challenge/index.shtml
[47] G. Arcas, H. Gonzales, and J. Cheng, "Challenge 7 of the Honeynet project forensic challenge 2011 - Forensic analysis of a compromised server," 2011. [Online]. Available: https://www.honeynet.org/challenges/forensic-challenge-7-analysis-of-a-%compromised-server/
[48] J. Zhu *et al.*, "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng.: Softw. Eng. Practice*, 2019, pp. 121–130.
[49] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng. Companion*, 2016, pp. 102–111.
[50] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *Proc. 37th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2007, pp. 575–584.
[51] R. Marty, A. Chuvakin, and S. Tricaud, "Challenge 5 of the Honeynet project forensic challenge 2010 - Log mysteries," 2010. [Online]. Available: https://www.honeynet.org/challenges/forensic-challenge-5–2010-log-myste%ries/
[52] F. Chollet *et al.*, "Keras," 2015. [Online]. Available: https://keras.io
[53] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Conf. Operating Syst. Des. Implementation*, 2016, pp. 265–283.
[54] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 559–563, 2017.
[55] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–15.
[56] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[57] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 33–40.
[58] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Towards automated log parsing for large-scale log data analysis," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 6, pp. 931–944, Nov./Dec. 2018.
[59] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, 2008, pp. 1322–1328.
[60] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
[61] M. R. Smith, T. Martinez, and C. Giraud-Carrier, "An instance level analysis of data complexity," *Mach. Learn.*, vol. 95, no. 2, pp. 225–256, 2014.
[62] J. Laurikkala, "Improving identification of difficult small classes by balancing class distribution," in *Proc. Conf. Artif. Intell. Medicine Eur.*, 2001, pp. 63–66.
[63] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2980–2988.
[64] O. Irsoy and C. Cardie, "Opinion mining with deep recurrent neural networks," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 720–728.
[65] H. Studiawan, C. Payne, and F. Sohel, "Graph clustering and anomaly detection of access control log for forensic purposes," *Digit. Invest.*, vol. 21, pp. 76–87, 2017.

**Hudan Studiawan** received the bachelor's and master's degrees from Institut Teknologi Sepuluh Nopember, Indonesia, in 2009 and 2011, respectively. He is currently working toward the PhD degree at Murdoch University, Australia. His current research interests are anomaly detection and machine learning.

**Ferdous Sohel** (Senior Member, IEEE) received the PhD degree from Monash University, Australia. He is currently an associate professor with Information Technology at Murdoch University, Australia. Prior to joining Murdoch University in 2015, he was a research assistant professor/research fellow at the School of Computer Science and Software Engineering, the University of Western Australia from January 2008 to mid-2015. His research interests include computer vision, image processing, pattern recognition, multimodal biometrics, scene understanding, robotics, and video coding. He is a recipient of prestigious Discovery Early Career Research Award (DECRA) funded by the Australian Research Council. He is a recipient of two WA state government funded competitive grants on shark hazard mitigation and digital pathology to improve cancer diagnosis. He is also a recipient of the VC's Early Career Investigators Award (UWA) and the Best PhD Thesis Medal from Monash University. He is an associate editor of *IEEE Transactions on Multimedia* (2019–2021) and *IEEE Signal Processing Letters* (2020–2021). He is a member of the Australian Computer Society.

**Christian Payne** received the PhD degree from Murdoch University, in 2009. He is an adjunct lecturer with the School of Engineering and Information Technology, Murdoch University, Australia. He has previously taught in the area of introductory programming and information security. His research interests include computer security, applied cryptography, and legal issues in technology.